BuildIT

PC Components Online Store — Mini-Capstone Project

Easy-to-Understand Reviewer

Node.js · Express · EJS · MySQL

A plain-English guide to how your project works,
from the database all the way to the checkout page.

Repository: `github.com/Jem4y/Mini-Capstone`
Main file: `app.js` | Server port: `3000`

Last updated: October 15, 2026 — includes the 12-part category update, working make-admin.js, and real product photos.

What's inside this reviewer

1. Project overview — what BuildIT is
2. The tech stack, explained simply
3. Project files and what each one does
4. Database design — the 7 tables
5. How a web request flows (step by step)
6. Feature walkthrough — Home, Login/Register, Cart, Checkout, OTP, PC Builder, Admin
7. Every route in the app
8. Security features and why they matter
9. Key functions in `app.js` — quick glossary
10. How to run the project
11. Likely defense questions with sample answers

1. Project overview

BuildIT is an online store for PC components (processors, graphics cards, motherboards, RAM) aimed at the Philippine market. Think of it as a small Lazada/Shopee, but specialized in computer parts, with its own **PC Builder** tool and a separate **admin dashboard**.

Two kinds of users

User	What they can do	Sign-in page
Customer	Browse products, build a PC, add to cart, checkout with OTP verification, view their order	`/login`
Admin	Add / edit / delete products, see dashboard stats, see the customer list	`/admin/login`

The main customer journey

1. Visitor opens the home page → sees categories and products.
2. Registers an account (or logs in).
3. Adds products to the cart (or builds a whole PC at once).
4. Goes to checkout → fills in a Philippine address and picks a payment method.
5. Enters a **6-digit OTP code** to approve the payment.
6. Order is saved, stock is reduced, and an order confirmation page is shown.

2. The tech stack, explained simply

Technology	Plain-English explanation
Node.js	Runs JavaScript on the server (not just in the browser).
Express	The web framework — it matches URLs (like `/cart`) to functions that handle them.
EJS (`views/*.ejs`)	HTML templates with little `<% %>` tags. Express fills them with data and sends the finished HTML to the browser.
MySQL + `mysql2`	The database. `mysql2` is the driver Node uses to talk to it.
express-session	Remembers who is logged in between requests using a session cookie.
bcrypt	Hashes passwords so only the hash is stored, never the real password.
dotenv	Reads secrets (DB password, session secret) from the `.env` file so they stay out of the code.
nodemon	Restarts the server automatically when you save a file (development tool).

> **Mental model:** Browser ⇄ *Express routes* (app.js) ⇢ *EJS views* (what the user sees) ⇄ *MySQL database* (the memory).

3. Project files and what each one does

File / folder	Purpose
`app.js`	**The heart of the app.** Sets up Express, creates all database tables, and contains every route (home, login, cart, checkout, OTP, orders, admin).
`conn.js`	Creates the MySQL **connection pool** (up to 10 connections at once) and exports it. Reads credentials from `.env`.
`admin.js` (root)	Source copy of the admin page's front-end JavaScript: mobile sidebar, add/edit product dialog, discount calculator, live product search, confirm-before-delete. The browser actually loads the copy in `public/admin.js` — keep both in sync.
`nav.js`	Source copy of the navbar script (desktop dropdowns, mobile hamburger menu, Esc closes menus). The same code is inlined in `index.ejs`; the root file is kept as a reference.
`create-admin.js`	Terminal tool (`node create-admin.js`) that asks for name, email, password and inserts a bcrypt-hashed admin into the database. **This is the admin tool to use.**
`make-admin.js`	Promotion tool (`node make-admin.js email`) — turns an existing registered customer into an admin by copying their account into the `admins` table (where admin login checks). The password stays the same. If the email isn't registered yet, it tells you to register first — or use `create-admin.js` to create a brand-new admin account from scratch.
`admin-table.sql`	Optional SQL file that creates the `admins` table by hand (the app also auto-creates it on startup).
`views/`	EJS pages: `index`, `login`, `register`, `cart`, `checkout`, `otp`, `order-success`, `pcbuilder`, `admin`, `admin-login`, `admin-sidebar`.
`public/`	Static assets served directly: CSS files (`style.css`, `checkout.css`, `admin.css`, `login.css`, `register.css`, `pcbuilder.css`), the front-end JS files (`admin.js`), and **product photos** in `public/images/` — e.g. `case.png`, `PSU.jpeg`, `cooler.webp`, `mouse.webp`, `powercord.webp`, `rog.webp`, `ssd.webp`, `wooting.jpg` — plus the `no-image.svg` fallback.
`.env`	Secrets: `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `SESSION_SECRET`, `PORT`. **Never committed to git** (listed in `.gitignore`).
`package.json`	Lists dependencies; `npm start` runs `nodemon app.js`.

4. Database design — the 7 tables

The database is named `buildit`. On startup, `initDB()` in `app.js` creates every table automatically (`CREATE TABLE IF NOT EXISTS`), adds any missing columns to old `products` tables, and seeds demo data if the tables are empty. That is why a fresh clone works with zero manual SQL.

Table	What it stores	Key points

`users`	Customer accounts	Email is UNIQUE; password is a bcrypt hash.
`admins`	Admin accounts — **separate** from customers	Same structure, but different table so a customer can never become an admin by accident.
`categories`	The store's 12 PC-part categories: Case, Motherboard, CPU, RAM, Storage (SSD/HDD), GPU, PSU, Cooling System, Monitor, Keyboard, Mouse, Power Cable	Has icon (Bootstrap Icons), badge (HOT/SALE…), and a label field. Older databases are migrated automatically on startup.
`products`	The catalog	`price` and optional `oldPrice` (for discounts), `stock`, `image` URL, `badge`, `description`.
`cart_items`	What each user has in their cart	`UNIQUE (user_id, product_id)` → the same product gets its quantity increased instead of a new row. Foreign keys cascade on delete.
`orders`	Placed orders: full PH address, payment info, subtotal, shipping fee, total, status	**No card numbers or CVVs are stored** — only a reference like "Visa debit ending 4242".
`order_items`	The products inside each order	Stores a copy of the name and price *at the time of purchase*, so history stays correct even if the product is later edited or deleted.

Relationships (entity relationships)

```
users 1 ——— many cart_items many ——— 1 products
users 1 ——— many orders    orders 1 ——— many order_items
categories 1 ——(by name)—— many products
admins  (standalone — no links to customers)
```

Seed data & product images

If the tables are empty, the app inserts all **12 categories** and one sample product per category (Ryzen 7 7800X3D, RTX 4070 Super, ASUS ROG Strix B650-F, G.SKILL Trident Z5, plus a case, SSD, PSU, cooler, monitor, keyboard, mouse, and power cord). On existing databases, startup **renames the old category names** (Processors → CPU, Graphics Cards → GPU, Motherboards → Motherboard, RAM Memory → RAM) and adds any missing categories — so every install ends up with the same 12-part list.

A product's `image` can be an **external URL** (the original seeds use Unsplash) or a **local file under `public/images/`** referenced as `/images/case.png` — that's how the real product photos in this project are stored. Anything else is rejected by the server, and a missing image falls back to `/images/no-image.svg`. `description` and `oldPrice` (the strikethrough discount anchor) are optional.

5. How a web request flows (step by step)

Example: a logged-in customer clicks "Add to Cart" on the RTX 4070.

1. **Browser** sends `POST /cart/add` with `productId=2`.
2. **Session middleware** reads the session cookie and gives the request a `req.session`.
3. `requireLogin` checks `req.session.userId`. No user? Redirect to `/login`.
4. **The route handler** runs: it checks the product exists and has stock, then runs `INSERT INTO cart_items ... ON DUPLICATE KEY UPDATE quantity = quantity + 1`.
5. **Response:** `res.redirect('/cart')` → the browser requests `GET /cart`, which joins `cart_items` with `products`, totals it, and renders `cart.ejs`.
6. The browser receives finished HTML + CSS/JS from `public/` and displays the page.

> **Pattern used everywhere:** check permission → validate input → run a *parameterized* SQL query → redirect or render a view. Every handler also has a `try/catch` that logs the error and shows a friendly 500 message.

6. Feature walkthrough

6.1 Home page (`/`)

- Loads categories, then counts products *per category that are actually in stock* (`stock > 0`), so the numbers are always live.
- Filtering: `?category=Processors` — but the category is **checked against the database list**, so random values are ignored.
- Sorting: `?sort=low` (cheapest first) or `?sort=high` (most expensive first).
- Only in-stock products are shown to customers.

6.2 Registration & login

- Register checks: terms accepted, passwords match, email not already used.
- Password is hashed with `bcrypt.hash(password, 12)` — cost factor 12.
- Login uses `bcrypt.compare`; both "wrong email" and "wrong password" show the same message: "*Invalid email or password.*"
- On success the session stores only `userId` and `userName`.

6.3 Shopping cart

- `GET /cart` — joins cart rows with products, computes each line subtotal and the total.
- `POST /cart/add` — rejects out-of-stock products; same product increments quantity.
- `POST /cart/update` — increase/decrease. Increasing stops at the stock limit; decreasing below 1 deletes the row.
- `POST /cart/remove` — deletes the row. All three verify the cart row **belongs to the logged-in user**.

6.4 Checkout (Philippine-focused)

- Address fields: street, barangay, city, province, region (all 17 PH regions listed), and a 4-digit postal code.
- Phone must match `09XXXXXXXXX` or `+639XXXXXXXXX`; stored normalized to `09…`.
- **Shipping:** ₱150 for NCR, ₱250 elsewhere, **free for orders ₱30,000 and above**.
- Payment methods: *Debit Card*, *E-Wallet* (GCash, Maya, GrabPay, ShopeePay), or *Cash on Delivery*.
- Card validation: exactly 16 digits, passes the **Luhn checksum** (catches typos), expiry MM/YY not in the past, 3–4 digit CVV. The brand (Visa/Mastercard/JCB) is detected from the number pattern.
- **The card number and CVV are validated and then thrown away** — the order only stores "Visa debit ending 4242".

6.5 OTP payment verification

Every payment method must be approved with a one-time code before the order exists:

Rule	Value	Why
Code length	6 digits	Standard length, generated with `crypto.randomInt` (no bias).
Validity	5 minutes	Then it expires and a new one must be requested.
Max wrong tries	5	Then the code is burned and the checkout must restart.
Resend cooldown	30 seconds	Prevents spamming codes.
Storage	SHA-256 hash in the session	The real code is never stored; comparison is timing-safe.

The validated order form is stashed in `req.session.pendingCheckout` before redirecting to `/checkout/otp`. Only after the correct code is entered does `placeOrder()` run. Because there is no SMS gateway in a student project, the code is displayed on the page (and logged to the server console) as a *demo OTP*.

6.6 Placing the order — transactions

`placeOrder()` does everything inside **one MySQL transaction**:

1. `SELECT ... FOR UPDATE` locks the cart rows and products (nobody can change stock mid-order).
2. Verifies stock is still enough; if not, rollback + a helpful message ("Only 2 left…").
3. Recalculates subtotal **from database prices, never from the browser**.
4. Inserts the `orders` row and one `order_items` row per product.
5. Decreases each product's stock.
6. Deletes the cart, then `COMMIT`. Any error → `ROLLBACK`, so the database is never left half-updated.

6.7 PC Builder (`/pcbuild`)

- Has a dropdown for **all 12 parts**: Case, Motherboard, CPU, RAM, Storage (SSD/HDD), GPU, PSU, Cooling System, Monitor, Keyboard, Mouse, and Power Cable.
- Each dropdown only lists in-stock products of that part's category (one database query feeds all 12 lists).
- "Add rig to cart" (`POST /pcbuild/add-all`) validates every selected id against the database (must exist and be in stock), then inserts each component with the same `ON DUPLICATE KEY` logic used by normal add-to-cart.
- The live total adds up the `data-price` of every selected dropdown.

6.8 Admin dashboard (`/admin`)

- **Stats:** total products, total customers, low-stock items (≤ 5), and total value sitting in everyone's carts.
- **Products:** add / edit / delete through a dialog. Server-side validation checks the name, a real category, price ranges, whole-number stock, and that `oldPrice > price` for discounts.
- **Customers:** newest 100 accounts with how many items each has in their cart.
- **UX niceties:** flash messages ("Saved … to the database"), live search over the product table, confirm prompts before deleting, and a discount calculator in the dialog (type old price + 10% → new price is filled in automatically).
- Database errors are translated to plain English by `describeDbError()` (e.g., "Check that MySQL is running").

7. Every route in the app

Public & customer routes

Method	Path	Protection	What it does
GET	`/`	—	Home: categories + products, with filter and sort
GET	`/login`	—	Login page
POST	`/login`	—	Checks email + bcrypt password, starts the session
GET	`/register`	—	Registration page
POST	`/register`	—	Validates and creates the customer
GET	`/cart`	login	Cart page with totals
POST	`/cart/add`	login	Add a product (stock-checked)
POST	`/cart/update`	login	+/− quantity, capped by stock
POST	`/cart/remove`	login	Remove a cart row (owner-checked)
GET	`/pcbuild`	—	PC Builder page
POST	`/pcbuild/add-all`	login	Add the whole build to the cart
GET	`/checkout`	login + CSRF	Checkout form
POST	`/checkout`	login + CSRF	Validates the form → sends to OTP (or COD straight to order)
GET	`/checkout/otp`	login + CSRF	OTP entry page (issues a code if needed)
POST	`/checkout/otp`	login + CSRF	Verifies the code → places the order
POST	`/checkout/otp/resend`	login + CSRF	New code after the 30s cooldown
GET	`/orders/:id`	login	Order confirmation — **owner only** (`WHERE user_id = ?`)
GET	`/logout`	—	Destroys the session

Admin routes

Method	Path	Protection	What it does
GET	`/admin/login`	CSRF	Admin sign-in page
POST	`/admin/login`	CSRF	Verifies admin, regenerates the session ID
GET	`/admin/logout`	—	Clears `adminId`

GET	`/admin`	admin + CSRF	Dashboard: stats, products, customers
POST	`/admin/products`	admin + CSRF	Add product (validated)
POST	`/admin/products/:id`	admin + CSRF	Edit product (validated)
POST	`/admin/products/:id/delete`	admin + CSRF	Delete product

8. Security features and why they matter

Threat	How this project defends against it
Passwords leaked from the database	bcrypt hashes (cost 12). The database only ever stores the hash, so even if it leaks, passwords can't be read.
SQL injection	All user values go through **parameterized queries** (`?` placeholders via `db.execute`), so input can never become SQL code.
CSRF (tricking an admin into clicking a malicious link)	A per-session **CSRF token** is placed in every form (`_csrf`) and compared with `crypto.timingSafeEqual` on every POST. Checkout and all admin POSTs are covered.
Session fixation after login	Admin login calls `req.session.regenerate()` , giving a brand-new session ID when privileges change.
Timing attacks on codes/passwords	OTP and CSRF comparisons use `crypto.timingSafeEqual` ; admin login always runs a bcrypt compare (against a dummy hash) even when the email doesn't exist, so response time doesn't reveal valid emails.
OTP guessing / brute force	6-digit code via `crypto.randomInt` (unbiased), 5-minute expiry, 5 attempts max, 30-second resend cooldown, only the SHA-256 hash is stored.
Credit card data theft	Card number and CVV are validated, then **discarded**. Only the brand + last 4 digits are saved as a reference.
Broken/dirty data	Server-side validation of every admin product field and every checkout field; totals are recalculated from DB prices, so a tampered browser can't change what's charged.
Accessing other people's data	Cart actions filter by `user_id` ; `/orders/:id` requires the order to belong to the logged-in user; `requireAdmin` re-checks the `admins` table on every request instead of trusting the session blindly.
Secrets in the source code	`.env` + `dotenv` ; the app refuses to start without `SESSION_SECRET` ; `.env` is in `.gitignore` .
Race conditions on stock	Orders run in a transaction with `SELECT ... FOR UPDATE` row locks, so two simultaneous checkouts can't oversell a product.

9. Key functions in `app.js` — quick glossary

Function	One-line explanation
`initDB()`	Creates/upgrades all tables and seeds demo data on startup.
`requireLogin`	Middleware: no `userId` in the session → redirect to /login.
`requireAdmin`	Middleware: re-checks the admins table every request; clears stale sessions.
`csrfProtect`	Middleware: injects and verifies the per-session CSRF token on POSTs.

`shippingFor(region, subtotal)`	₱150 NCR / ₱250 elsewhere / free at ₱30k+.
`luhnValid()`	Checksum that catches mistyped card numbers.
`cardBrand()`	Guesses Visa/Mastercard/JCB from the number's first digits.
`validateCheckout()`	Cleans + checks every checkout field; returns the safe form and payment info.
`loadCart()`	One shared query for "the user's cart joined with products".
`placeOrder()`	The transactional order writer (lock → check → insert → decrement → clear cart → commit).
`generateOtp()`	Unbiased 6-digit code from Node's crypto.
`hashOtp()` / `otpMatches()`	SHA-256 hashing + constant-time comparison of the code.
`issueOtp()`	Puts a fresh code (hash + expiry + attempt counter) into the session.
`renderCheckout()` / `renderOtp()`	Shared page-render helpers (redirect empty carts, compute totals).
`parseProductForm()`	Admin-side product validation with friendly error messages.
`describeDbError()`	Turns cryptic MySQL errors into human instructions.
`dbAction()`	Wrapper that catches any error in an admin action and flashes it.
`setFlash()`	One-shot success/error message shown after a redirect.
`orderNumber()`	Makes pretty order numbers like `BIT-000042`.

10. How to run the project

```
# 1. Requirements: Node.js and MySQL running locally

# 2. Install dependencies
npm install

# 3. Create the .env file (already present in this setup)
#    DB_HOST=localhost
#    DB_USER=root
#    DB_PASSWORD=your_mysql_password
#    DB_NAME=buildit
#    SESSION_SECRET=some_long_random_string
#    PORT=3000

# 4. Start the app (tables + demo data are created automatically)
npm start

# 5. Open the store
#    Customer:  http://localhost:3000
#    Admin:     http://localhost:3000/admin/login

# 6. Create your admin account (one time only)
node create-admin.js
```

> **No SQL needed:** just start the app once and every table exists. `admin-table.sql` is only there as a backup / for showing the schema.

11. Likely defense questions with sample answers

Q: What is this project in one sentence?

A: BuildIT is a full-stack e-commerce web app for PC components, built with Node.js, Express, EJS, and MySQL, featuring a PC builder, a OTP-secured Philippine checkout flow, and a separate admin dashboard.

Q: How are passwords stored?

A: Hashed with bcrypt using cost factor 12. Plain-text passwords never touch the database, and login uses `bcrypt.compare()` against the stored hash.

Q: How do you prevent SQL injection?

A: Every query uses placeholders (`?`) through `mysql2`'s `execute()`, so user input is always sent as data, never as SQL code.

Q: What is CSRF and how do you handle it?

A: Cross-Site Request Forgery tricks a logged-in user into sending a fake request. Each session gets a random token that is embedded in every protected form and verified with a constant-time comparison before any POST is allowed to run.

Q: What does the OTP step do? What stops someone from guessing it?

A: After checkout validation, a 6-digit code (from `crypto.randomInt`, no bias) is generated and hashed with SHA-256 into the session. It expires after 5 minutes, locks after 5 wrong attempts, and resend requests are throttled to one per 30 seconds.

Q: Do you store credit card numbers?

A: No. Card details are validated in memory (16 digits, Luhn checksum, expiry, CVV) and then discarded. Only a reference like "Visa debit ending 4242" is saved with the order.

Q: Why is `oldPrice` higher than `price` enforced?

A: It guarantees the discount badge shown to customers is honest — an admin can't fake a "sale" by raising the price first. The check happens both in the admin dialog (client-side hint) and in `parseProductForm()` on the server.

Q: What happens if two customers buy the last item at the same time?

A: The order runs in a database transaction with `SELECT ... FOR UPDATE` row locks. The first order commits and reduces the stock; the second one sees stock = 0, rolls back, and the customer is told the item just went out of stock.

Q: Why are admins in a separate table from users?

A: Separation of privileges. Signing up as a customer can never accidentally grant admin rights, and admin routes additionally re-verify the account in the `admins` table on every request.

Q: Why recalculate totals on the server instead of trusting the form?

A: The browser can be manipulated (DevTools or intercepting proxies). Prices, subtotal, shipping, and total are always recomputed from database values inside `placeOrder()`.

Q: How are product images handled?

A: The `products.image` column stores either a full `http(s)://` URL or a local path like `/images/case.png` that maps to a file in `public/images/`. `parseProductForm()` rejects anything else, and the catalog falls back to `/images/no-image.svg` when no image is set.

Q: What is the PC Builder and how does it work?

A: A page with 12 dropdowns — one for every part on the parts list (Case, Motherboard, CPU, RAM, Storage, GPU, PSU, Cooling System, Monitor, Keyboard, Mouse, Power Cable) — each listing only in-stock products of that category. "Add rig to cart" validates the selected ids and inserts them into `cart_items`, reusing the same duplicate-key increment logic as normal add-to-cart.

Q: How does the app set up its database?

A: `initDB()` runs on startup: `CREATE TABLE IF NOT EXISTS` for all seven tables, an `ALTER TABLE` pass that adds any missing columns to older databases, a category upgrade that renames legacy category names (e.g. Processors → CPU) and adds any missing parts from the 12-item list, and seed inserts only when the tables are empty. So the app is self-configuring.

Q: What are the known limitations?

A (honest answers): the OTP is displayed on-screen/console because there's no SMS or e-mail gateway; payment is simulated (no real gateway like PayMongo/Stripe); there's no password-reset e-mail; and orders don't yet have a full status-tracking UI for the customer.

Generated from the source code in `/Mini-Capstone` — app.js, conn.js, admin.js, nav.js, create-admin.js, admin-table.sql, and the EJS views. Use it to review before a demo or a defense.